



Ho Chi Minh City University of Technology
Faculty of Computer Science and Engineering

Chapter 7: NP-completeness

Algorithms Analysis and Design (C03031)

Instructor: Assoc. Prof. Dr. Vo Thi Ngoc Chau

Email: chauvtn@hcmut.edu.vn



Course Learning Outcomes

1. Able to analyze the complexity of the algorithms (recursive or iterative) and estimate the efficiency of the algorithms.
2. Improve the ability to design algorithms in different areas *by decomposing a computing problem into advanced algorithm design strategies.*
3. Able to discuss NP-completeness.



References

- [1] Cormen, T. H., Leiserson, C. E, and Rivest, R. L., *Introduction to Algorithms*, The MIT Press, 2009.
- [2] Levitin, A., *Introduction to the Design and Analysis of Algorithms*, 3rd Edition, Pearson, 2012.
- [3] Sedgewick, R., *Algorithms in C++*, Addison-Wesley, 1998.
- [4] Weiss, M.A., *Data Structures and Algorithm Analysis in C*, The Benjamin/Cummings Publishing, 1993.



Course Outline

- ❑ Chapter 1. Fundamentals
- ❑ Chapter 2. Divide-and-Conquer Strategy
- ❑ Chapter 3. Decrease-and-Conquer Strategy
- ❑ Chapter 4. Transform-and-Conquer Strategy
- ❑ Chapter 5. Dynamic Programming and Greedy Strategies
- ❑ Chapter 6. Backtracking and Branch-and-Bound
- ❑ **Chapter 7. NP-completeness**
- ❑ Chapter 8. Approximation Algorithms



Chapter 7.

NP-completeness

- 7.1. Deterministic and Nondeterministic Polynomial-time algorithms
- 7.2. NP-completeness
- 7.3. Cook's theorem
- 7.4. The halting problem
- 7.5. Some NP-complete problems
- 7.6. Some approaches to cope with NP-complete problems



Quick Review

- *Problem* vs. *Instance* of a problem
 - Sorting an array of $\{1, 6, 3, 8, 9, 2, 0, 5\}$
 - Generating permutations from an array of $\{a_1, a_2, a_3, a_4\}$
 - Efficiency class of *algorithms* for different problems
 - Sorting n elements: $O(n \log_2 n)$ with Heapsort, $O(n^2)$ with Insertion sort
 - Permutations of n elements: $O(n!)$ with PERM
- Some problems can be solved *efficiently*, some *inefficiently*, and some *cannot*. Why?



Quick Review

- For many problems we have several efficient algorithms to solve.
 - Sorting n elements, search for a key in n elements, ...
- However, many problems arise in practice which do not have such efficient solving algorithms.
 - Permutations of n elements, graph coloring, bin packing, ...
- For a large class of problems we can't even tell *whether or not an efficient algorithm might exist*.
 - Hamiltonian cycle problem, traveling salesman problem, ...



Quick Review

□ Easy problems vs. Hard problems

- A great deal of research has been done in this area and lends to the development of *mechanisms* by which new problems can be classified as being *as difficult as* old problems.
- Sometimes, the line between “easy” and “hard” problems is hardly decided.

For example:

- P1 (*Easy*): Is there a path from x to y with weight $\leq M$?
- P2 (*Hard*): Is there a path from x to y with weight $\geq M$?

P1 – BFS in *linear* time

P2 – Branch-and-Bound in *exponential* time



7.1. Deterministic and Nondeterministic Polynomial-Time algorithms

P: a set of all the *problems* that can be solved by *deterministic algorithms* in *polynomial* time.

Deterministic: at any time, whatever the algorithm is doing, there is only one thing that it could do next.

For example: *Sorting N elements* belongs to P since there is *Insertion sort* whose time complexity is proportional to N^2 .



7.1. Deterministic and Nondeterministic Polynomial-Time algorithms

□ Non-determinism

- One way to extend the power of a computer is to endow it with the power of *non-determinism*.
- *Non-determinism*: when an algorithm is faced with a choice of several options, it has the power to “*guess*” the right one.

□ *Non-deterministic algorithms*

For example: Let A be an unsorted set of n positive integers. A nondeterministic algorithm $\text{NSORT}(A, n)$ sorts these numbers in ascending order and then outputs them in that order.



7.1. Deterministic and Nondeterministic Polynomial-Time algorithms

□ NSORT (A, n): a non-deterministic algorithm

// An array B is used as a temporary array.

procedure NSORT (A, n)

// sort n positive integers in ascending order: $\leq$

begin

for $i := 1$ **to** n **do** $B[i] := 0$;

 // *guessing stage*

for $i := 1$ **to** n **do**

begin

$j := \text{choice}(1:n)$;

if $B[j] \neq 0$ **then** failure

else $B[j] := A[i]$

end;

 // *verification stage*

for $i := 1$ **to** $n-1$ **do**

if $B[i] > B[i+1]$ **then** failure;

 print(B);

 success

end;

Function $\text{choice}(1:n)$ can choose one of the correct positions in the range from 1 to n .



7.1. Deterministic and Nondeterministic Polynomial-Time algorithms

- A deterministic interpretation of a nondeterministic algorithm can be made by allowing *unbounded parallelism* in computation.
- Whenever a choice is made, the algorithm makes *several copies of itself*. One copy is made for each of the possible choices. Thus, many copies are executing at the same time.
 - The first copy that reaches a successful completion terminates all of the other computations.
 - If a copy reaches a failure completion, then only that copy of the algorithm terminates.



7.1. Deterministic and Nondeterministic Polynomial-Time algorithms

□ Nondeterministic algorithms

- In fact, a nondeterministic machine does not make any copies of an algorithm when a choice is made.
- *Instead, it has the ability to select the “correct” element from a set of allowable choices when a choice is made.*
- A “correct” element is defined relatively to *the shortest sequence of choices* that leads to a successful termination.
- In case no sequence of choices leads to a successful termination, we shall assume that the algorithm terminates in one unit of time with output “*unsuccessful computation*”.



7.1. Deterministic and Nondeterministic Polynomial-Time algorithms

Note on the nondeterministic algorithm NSORT:

- The *success* and *failure* signals are equivalent to a *stop* statement in a deterministic algorithm.
- The complexity of NSORT is $O(n)$. Its *verification* stage runs in *polynomial time*.

NP: a set of all the problems that can be solved by *nondeterministic* algorithms in *polynomial time*.

For example: “Is there a longest path from vertex x to vertex y in a graph?” is an NP problem.



7.1. Deterministic and Nondeterministic Polynomial-Time algorithms

□ Circuit satisfiability problem (CSP)

Given a logical formula in the form of:

$$(x_1 + x_3 + x_5) * (x_1 + \sim x_2 + x_4) * (\sim x_3 + x_4 + x_5) * (x_2 + \sim x_3 + x_5)$$

where the x_i 's represent Boolean variables (*true* or *false*), $+$ represents OR, $*$ represents AND, and $\sim$ represents NOT.

The CSP problem is to determine whether or not there exists an assignment of truth values to the variables that makes the formula *true*.

CSP is an NP problem.



7.1. Deterministic and Nondeterministic Polynomial-Time algorithms

- A nondeterministic algorithm for CSP

```
procedure EVAL(E, n)
boolean x[n];
begin
  for i:= 1 to n do
    x[i] := choice(true, false)
  end;
  if (E is true) then success
  else failure
end;
```




7.1. Deterministic and Nondeterministic Polynomial-Time algorithms

Your turn:

- ❑ 7.1.1. Distinguish a deterministic algorithm from a nondeterministic polynomial-time one.
- ❑ 7.1.2. Which statement is correct about NP?
 - 7.1.2.1. NP is a set of the nonpolynomial time problems.
 - 7.1.2.2. NP is a set of the nondeterministic problems.
 - 7.1.2.3. NP is a set of the nonpolynomial time algorithms for hard problems.
 - 7.1.2.4. NP is a set of the problems that can be solved by nondeterministic algorithms in polynomial time.
 - 7.1.2.5. NP is a set of the hard problems that can be solved by deterministic algorithms in nonpolynomial time.
- ❑ 7.1.3. Is P a proper subset of NP? Why? Why not?



7.1. Deterministic and Nondeterministic Polynomial-Time algorithms

Your turn:

- ▣ 7.1.4. Given a decision version of the 0-1 knapsack problem as follows.

Given a knapsack of size M , a list of n items, each of which has weight w_i and value v_i , for $i=1..n$, and R is a positive real value. The problem is to check which items will be taken by the thief to obtain the total value of the knapsack greater than or equal to R .

Design and analyze a nondeterministic algorithm for this decision problem.



7.1. Deterministic and Nondeterministic Polynomial-Time algorithms

Your turn:

- ▣ 7.1.5. Design and analyze a nondeterministic algorithm for the subset sum problem.
- ▣ 7.1.6. Design and analyze a nondeterministic algorithm for the vertex cover problem.
- ▣ 7.1.7. Design and analyze a nondeterministic algorithm for the Hamiltonian cycle problem.

Hint: a decision version of each problem in 7.1.5, 7.1.6, and 7.1.7 might be defined and then a nondeterministic algorithm can be designed.

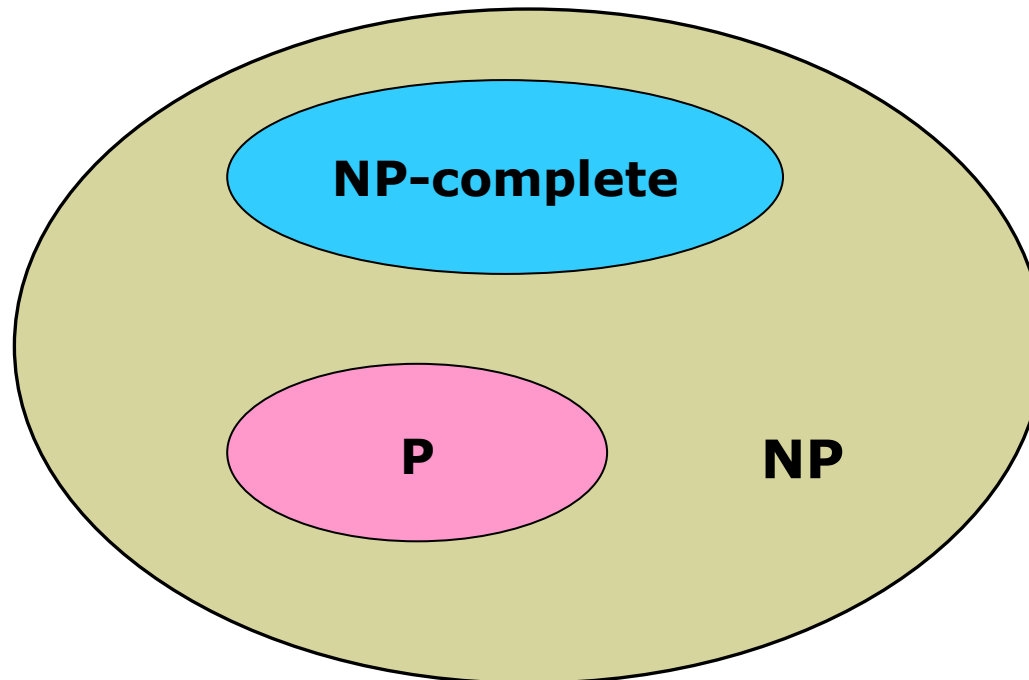


7.2. NP-completeness

- There is a list of problems that are known to belong to NP but might or might not belong to P. (*That is, they are easy to solve on a non-deterministic machine but, no one has been able to find an efficient algorithm on a conventional deterministic machine for any of them*).
- These problems have an additional property:
“If any of these problems can be solved in polynomial time on a deterministic machine, then so can all problems in NP.”

7.2. NP-completeness

- The problems that are known to belong to *NP* but *might or might not belong to P* are said to be *NP-complete*.





7.2. NP-completeness

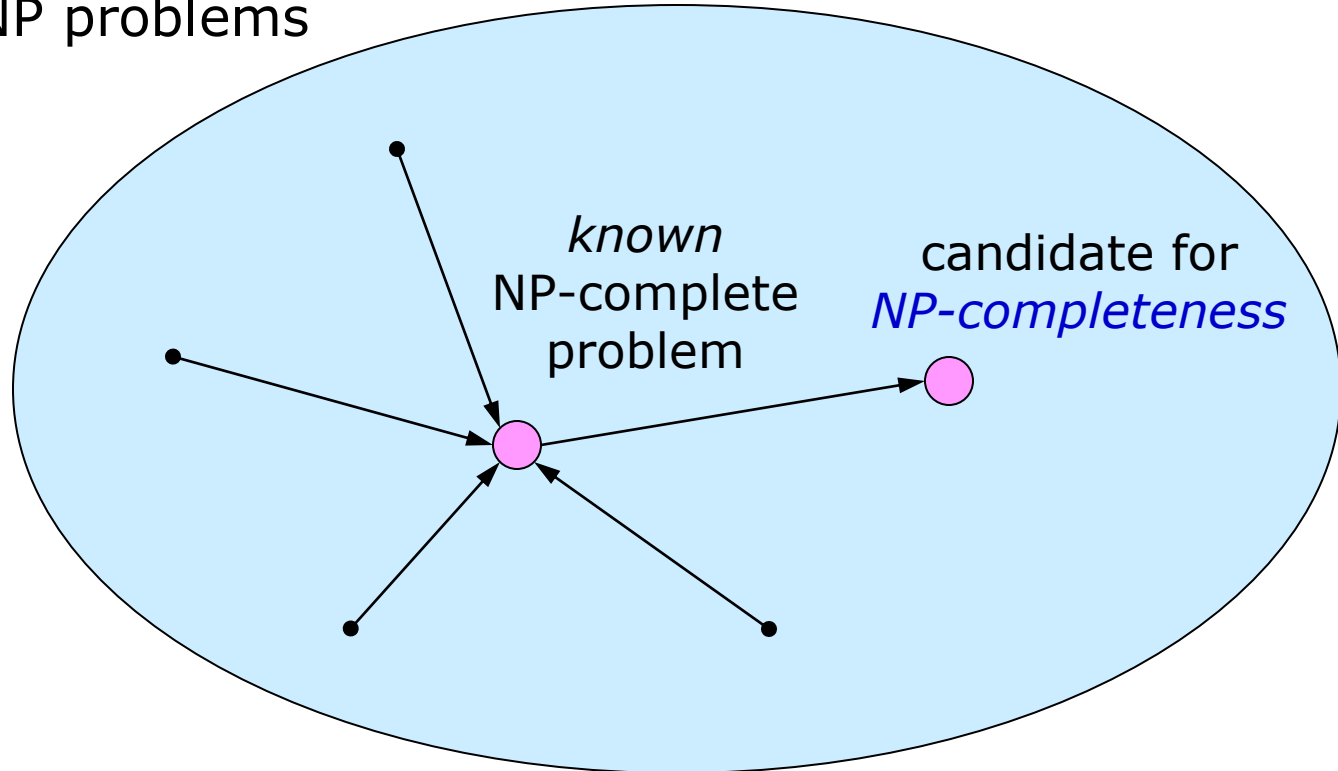
- NP-complete problems form a subset of the most difficult problems in NP.
- The primary tool used to prove that problems are NP-complete uses the idea of *polynomial reducibility*.
- Any algorithm to solve a new problem in NP can be used to solve some *known* NP-complete problem by the following process:

transform any instance of the known NP-complete problem to an instance of the new problem, solve the new problem using the given algorithm, then transform the solution back to a solution of the NP-complete problem.

7.2. NP-completeness

□ Proving NP-completeness by reduction

NP problems



7.2. NP-completeness

□ *Polynomial reducibility*

- Let $L1$ and $L2$ be problems.

$L1$ *polynomially reduces to* $L2$, (also written $L1 \alpha_p L2$)

if and only if there is a way to *transform $L1$ to $L2$* such that any deterministic algorithm that solves $L2$ in polynomial time can be used to solve $L1$.

- The transformation must be done in *polynomial* time.



7.2. NP-completeness

- ❑ To prove that a problem L is NP-complete, we need to prove that:
 1. L belongs to *NP*.
 2. Some known NP-complete problem is *polynomially reducible* to L .

For example: Given two following problems

- ❑ **Traveling Salesman Problem (TSP):** Given a set of cities and distances between a pair, find a tour of all the cities of distance less than or equal to M .
- ❑ **Hamiltonian Cycle Problem (HCP):** Given a graph, find a simple cycle that includes all the vertices.



7.2. NP-completeness

□ Given *HCP is NP-complete*, we wish to determine whether or not TSP is also NP-complete.

- TSP belongs to *NP*.
- Any algorithm for solving TSP can be used to solve HCP, via *polynomial reduction*:

Given an instance of HCP (a graph), construct an instance of TSP as follows:

- for cities in TSP, use the set of N vertices in the graph.
- for distances between each pair of cities, use **1** if there is an edge between the corresponding vertices in the graph, **2** if there is no edge.

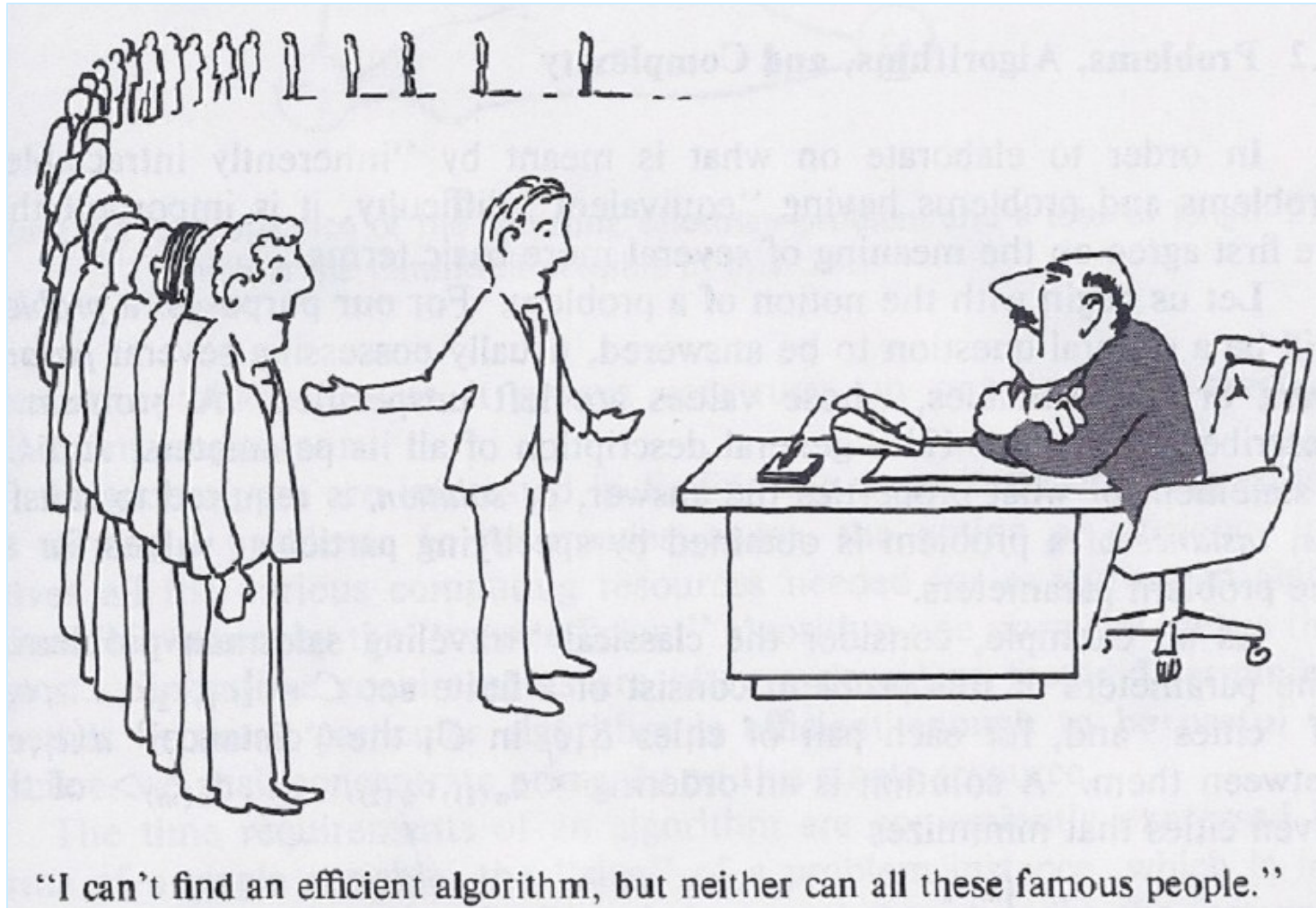
Use the algorithm for TSP to find a tour of distance $\leq N$. The tour corresponds precisely to the Hamiltonian cycle.



7.2. NP-completeness

- That is *HCP reduces to TSP*, so the NP-completeness of HCP implies the NP-completeness of TSP.
- The reduction of HCP to TSP is relatively simple because the problems are so similar.
- Actually, polynomial reductions can be quite complicated and can connect problems which seem to be quite dissimilar.
- Similarly, it is possible to reduce the circuit satisfiability problem to HCP.

7.2. NP-completeness



M. Garey and D.S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W.H. Freeman and Company, San Francisco, CA, 1979, pp.3.



7.2. NP-completeness

Your turn:

- ❑ 7.2.1. What is a NP-complete problem? Give an example.
- ❑ 7.2.2. List 3 reasons for a need to comprehend NP-completeness. Give an example for each reason.
- ❑ 7.2.3. The additional property of NP-complete problems is: “If any of these problems can be solved in polynomial time on a deterministic machine, then so can all problems in NP.” Which is implied from this property? Why?
 - 7.2.3.1. $P = NP$
 - 7.2.3.2. $P \subset NP$
 - 7.2.3.3. $P \supset NP$
 - 7.2.3.4. NP does not exist.



7.2. NP-completeness

Your turn:

- ▣ 7.2.3. Given the *Hamiltonian cycle* problem is NP-complete, prove that the *longest path* problem is also NP-complete.
 - The *Hamiltonian cycle* problem: determine whether a given graph $G=(V, E)$ has a Hamiltonian cycle.
 - The *longest path* problem: determine whether a given graph $G=(V, E)$ has a simple path of length greater than or equal to K where K is a positive integer such that $K \leq |V|$.



7.2. NP-completeness

Your turn:

□ 7.2.4. Given the *vertex cover* problem is NP-complete, prove that the *set cover* problem is also NP-complete.

- The *vertex cover* problem: for a given graph $G = (V, E)$ and a positive integer $m \leq |V|$, determine whether there is a vertex cover of size m or less for G . (A vertex cover of size k for a graph $G = (V, E)$ is a subset $V' \subseteq V$ such that $|V'| = k$ and, for each edge $(u, v) \in E$, at least one of u and v belongs to V' .)
- The *set cover* problem: given a finite set X and a family F of subsets of X , such that every element of X belongs to at least one subset in F : $X = \bigcup_{S \in F} S$ and a positive integer $m \leq |F|$,

$$S \in F$$

determine whether there is a subset $C \subseteq F$ whose members cover all of X : $X = \bigcup_{S \in C} S$ and $|C| \leq m$.

$$S \in C$$



7.2. NP-completeness

Your turn:

- 7.2.5. Given the clique problem is NP-complete, prove that the vertex cover problem is also NP-complete.
 - The clique problem: for a given graph $G = (V, E)$ and a positive integer $m \leq |V|$, determine whether G contains a clique of size m or more. (A clique of size k in a graph is its complete subgraph of k vertices.)
 - The vertex cover problem: for a given graph $G = (V, E)$ and a positive integer $m \leq |V|$, determine whether there is a vertex cover of size m or less for G . (A vertex cover of size k for a graph $G = (V, E)$ is a subset $V' \subseteq V$ such that $|V'| = k$ and, for each edge $(u, v) \in E$, at least one of u and v belongs to V' .)



7.3. Cook's theorem

- But: How was the *first* problem proven to be NP-complete?
- The *satisfiability* problem (SAT) is to determine whether a formula is true for some assignment of truth values to the variables.
 - CSP = 3-CNF SAT

S.A. Cook (1971) gave a direct proof that SAT problem is NP-complete.

“If there is a polynomial time algorithm for the satisfiability problem, then all problems in NP can be solved in polynomial time.”

The proof is extremely complicated. It is based on a general-purpose computer known as a Turing machine.



7.4. The halting problem

- The *halting* problem
 - a specific problem that is *algorithmically unsolvable*
 - an *undecidable* problem given by the English logician and computer science pioneer *Alan Turing* (1912-1954) in 1936
 - Defined:

“Given a computer *program* and an *input* to it, determine whether the program will *halt* on that input or continue working indefinitely on it.”



7.4. The halting problem

- A proof that there is no algorithm for the halting problem is made by *contradiction*.
 - Assume that *A is an algorithm* that solves the halting problem. That is, for any program *P* and input *I*,

$$A(P, I) = \begin{cases} 1, & \text{if program } P \text{ halts on } I; \\ 0, & \text{if program } P \text{ does not halt on } I. \end{cases}$$

- We can consider program *P* as an input to itself and use the output of algorithm *A* for pair (P, P) to construct a program *Q* as follows:

$$Q(P) = \begin{cases} \text{halts,} & \text{if } A(P, P) = 0, \text{ i.e., program } P \text{ does not halt on input } P; \\ \text{does not halt,} & \text{if } A(P, P) = 1, \text{ i.e., program } P \text{ halts on input } P. \end{cases}$$



7.4. The halting problem

- A proof that there is no algorithm for the halting problem is made by *contradiction*.
 - Then on substituting Q for P , we obtain

$$Q(Q) = \begin{cases} \text{halts,} & \text{if } A(Q, Q) = 0, \text{ i.e., program } Q \text{ does not halt on input } Q; \\ \text{does not halt,} & \text{if } A(Q, Q) = 1, \text{ i.e., program } Q \text{ halts on input } Q. \end{cases}$$

- This is a *contradiction* because neither of the two outcomes for program Q is possible.
- Algorithm A *does not* solve the halting problem.
- The halting problem is *unsolvable*.



7.4. The halting problem

□ The *halting* problem

“Given a computer *program* and an *input* to it, determine whether the program will *halt* on that input or continue working indefinitely on it.”

- The proof of the *undecidability* of the halting problem uses the *diagonalization* discovered by the German mathematician Georg Cantor in 1873.

- M. Sipser. *Introduction to the theory of computation*. 2nd Edition. Thomson Course Technology, 2006, pp. 173-181.



7.5. Some NP-complete problems

- Thousands of diverse problems are known to be NP-complete. Of course, the list begins with the *satisfiability* problem and includes *TSP*, the *Hamiltonian cycle* problem, as well as the *longest-path* problem.
- The following additional problems are representative:
 - ***Partition***: Given a set of integers, can they be divided into two sets whose sums are equal?
 - ***Integer linear programming***: Given a linear problem, is there a solution in integers?

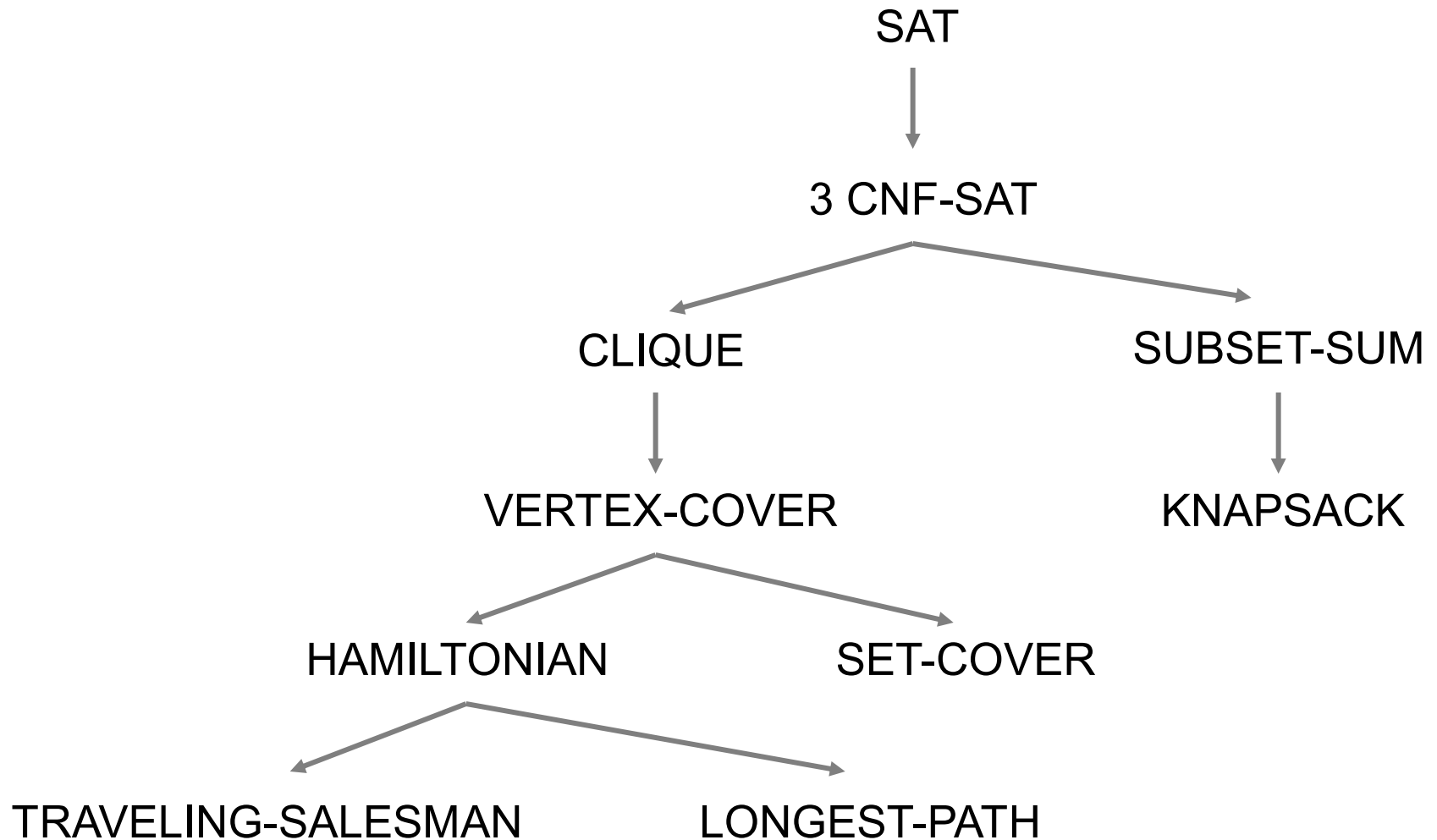


7.5. Some NP-complete problems

- ***Multiprocessor scheduling***: Given a deadline and a set of tasks of varying length to be performed on two identical processors, can the tasks be arranged so that the deadline is met?
- ***Vertex cover***: Given a graph and an integer N , is there a set of fewer than N vertices which touch all the edges?
- ***Bin packing***: We are given n objects which have to be placed in bins of equal capacity L . Object i requires l_i units of bin capacity. The objective is to determine the minimum number of bins needed to accommodate all n objects.



7.5. Some NP-complete problems



Relationship between some problems for polynomial reduction



7.5. Some NP-complete problems

$$P \neq NP ?$$

- These and many related problems have important natural practical applications.
- The fact that no good algorithm has been found for any of these problems is surely strong evidence that $P \neq NP$.
- Whether or not $P = NP$, the practical fact is that *we have at present no algorithms guaranteed to solve any of the NP-complete problems efficiently.*



7.6. Some approaches to cope with NP-complete problems

- ❑ Use an *approximation algorithm* that is not the best solution but a near-optimal solution.
- ❑ Based on “average-time” performance, develop an algorithm that finds the solution *in some cases*, but does not necessarily work in all cases.
- ❑ Use with “*efficient*” *exponential* algorithms, for example, *backtracking* or *branch-and-bound* algorithms.
- ❑ Invent *heuristics* and add it to the algorithm to improve the performance of the algorithm.
- ❑ Apply *metaheuristics*.



7.6. Some approaches to cope with NP-complete problems

- *Heuristics*: knowledge in a specific problem that can be used to guide the search process in finding a solution of an algorithm. Thanks to heuristics, the algorithm becomes more efficient.
- *Metaheuristics*: a kind of more general heuristics which can be used for solving several kinds of problems.
- Recently, metaheuristics is an active research field with the introduction of many popular metaheuristics, such as:
 - genetic algorithm
 - simulated annealing
 - Tabu search
 - etc.



7.6. Some approaches to cope with NP-complete problems

- There are several NP-complete problems in some following fields:
 - numerical analysis
 - geometry modeling
 - graph processing
 - image processing
 - data mining
 - ...
- The most important practical contribution of the theory of NP-completeness is that: *NP-completeness provides a mechanism to discover a new problem from any of many diverse areas is “easy” or “hard”*.



7.6. Some approaches to cope with NP-complete problems

- *Undecidable* problems
 - The **halting** problem
- *Intractable* problems
 - Generating all the permutations of n elements
- *NP-complete* problems
 - Traveling salesman problem
 - Hamiltonian cycle problem
- **P** problems
 - String matching on text T and pattern P
 - Sorting n elements



Your turn on 7.3-7.6

- (1). What is the first proven NP-complete problem?
- (2). What is implied from the Cook's theorem?
- (3). Which statement is correct about the halting problem?
 - (3.1). There is no efficient algorithm for the halting problem.
 - (3.2). There is no algorithm for the halting problem in polynomial time.
 - (3.3). There is no algorithm for the halting problem.
 - (3.4). There is no deterministic algorithm for the halting problem.



Your turn on 7.3-7.6

- (4). State ten NP-complete problems.
- (5). List three approaches dealing with NP-complete problems. For each listed approach, present one algorithm example for a particular NP-complete problem.
- (6). A *branch-and-bound* deterministic algorithm is designed for the *traveling salesman* problem. What is assumed about the problem when this algorithm is used in practice?



Algorithms Analysis and Design (C03031)

□ Chapter 7. NP-completeness



- 7.1. Deterministic and Nondeterministic Polynomial-time algorithms
- 7.2. NP-completeness
- 7.3. Cook's theorem
- 7.4. The halting problem
- 7.5. Some NP-complete problems
- 7.6. Some approaches to cope with NP-complete problems